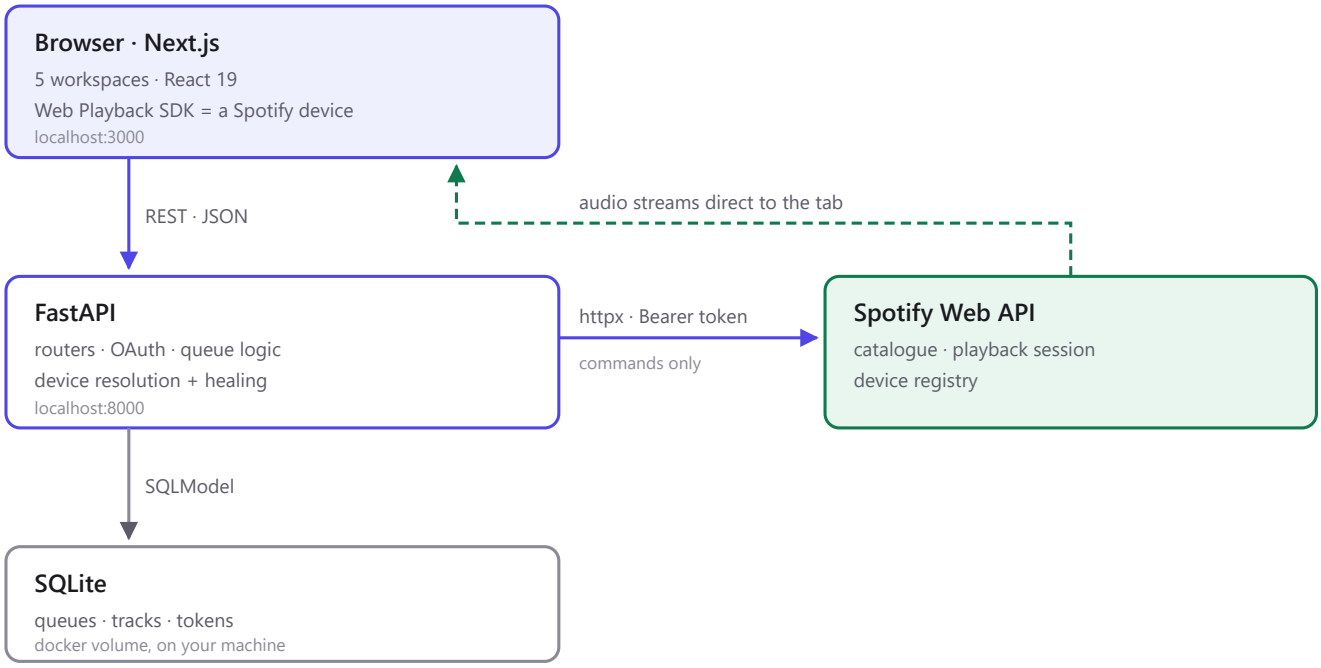
FOCUSLAB · REFERENCE

Spotify Architecture & Security

How playback works without the Spotify app, and where the trust boundaries sit.

01 System architecture

Four moving parts. The key asymmetry: **commands and audio travel different paths** — the backend issues instructions but never carries a single byte of audio, which streams straight from Spotify to the browser tab.



Control plane (solid) and audio plane (dashed) are entirely separate.

Why the browser is the device

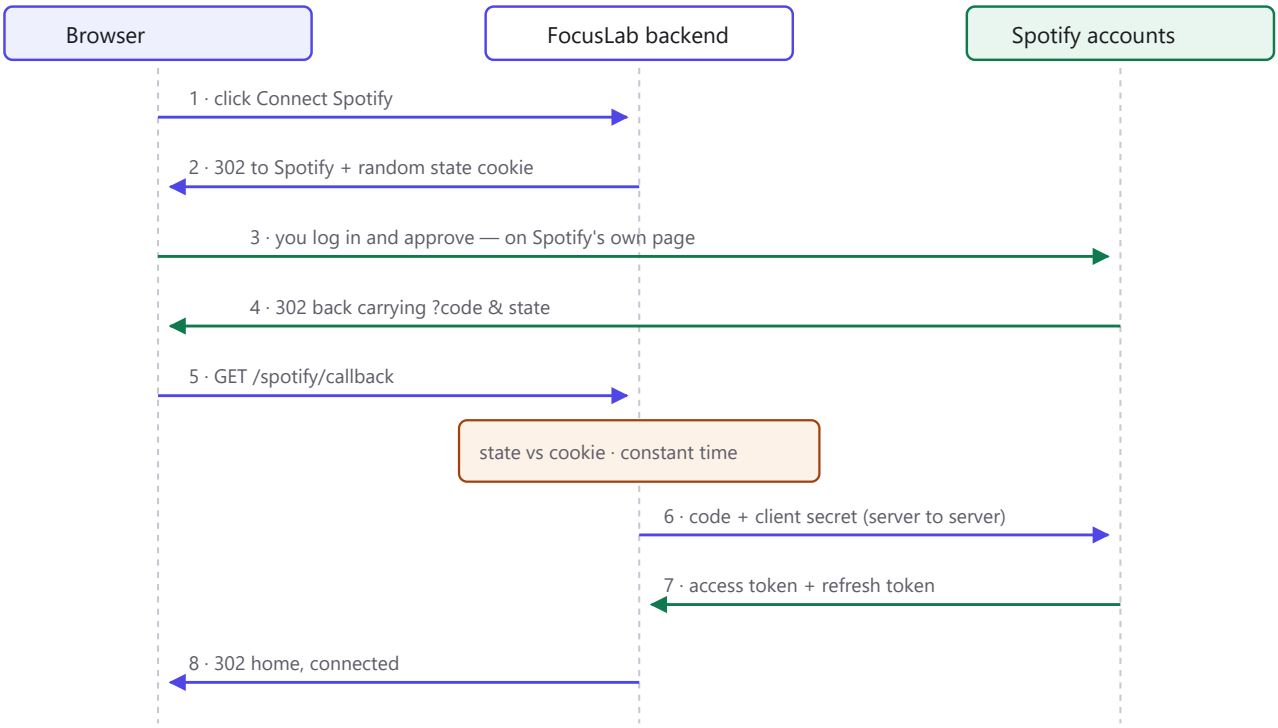
The Web Playback SDK registers the tab itself in Spotify's device registry. It appears on your phone like any speaker would, and no Spotify client needs to be running anywhere.

Layered access — one concern per level

Every playback route funnels through this — which is why the device fix touched one function, not five routes.

02 OAuth 2.0 — connecting an account

FocusLab never sees your Spotify password. It receives a limited, expiring permission slip instead.



Step 6 is the pivot: the code alone is worthless without the client secret, which never leaves the server.

Token lifecycle

Refreshing early closes the race where a token is valid at check time but expired on arrival.

03 The device model

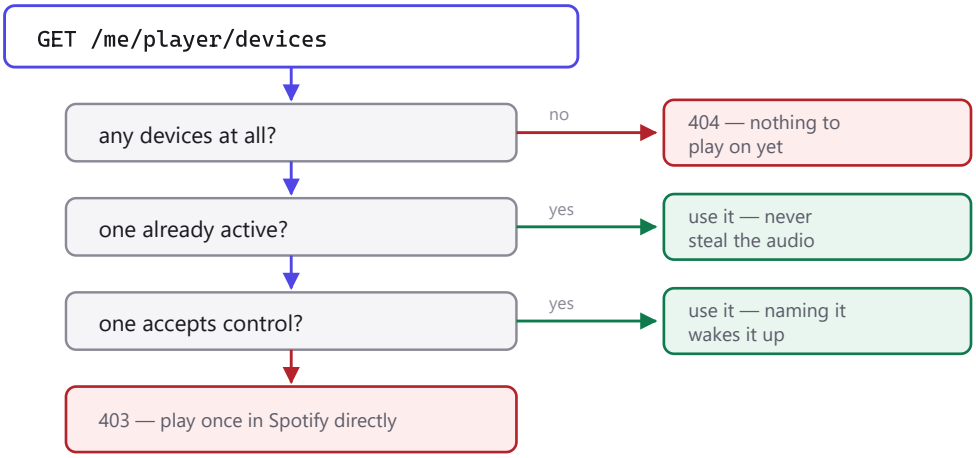
Spotify plays nothing itself — it forwards commands to a **device**. This single distinction caused the original failure:

AVAILABLE
Signed in and reachable.
Appears in the device list. Cannot be commanded.

ACTIVE
Currently holds the playback session.
Only one at a time. This is what commands default to.

The bug: opening Spotify is not enough
An open, idle client is *available* but not *active*. Commands sent with no `device_id` target "the currently active device" — there wasn't one — so Spotify answered `404` with a device sitting right there.

How a device is now chosen



Self-healing the SDK reconnects under a NEW id on every reload, so callers hold dead ids.
A 404 on a caller-supplied id re-resolves once and retries — guarded so it cannot loop.

04 Error translation

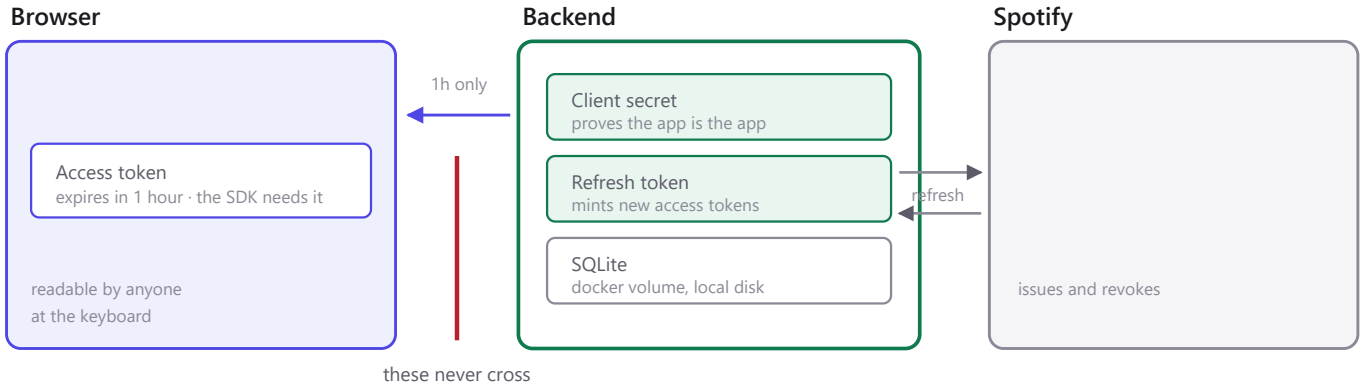
Spotify's vocabulary is not the UI's vocabulary. Each upstream failure maps to a code that means what it says.

SPOTIFY SAYS	WE RETURN	WHY
`401`	`401`	Token went bad — reconnect
`403` Restriction violated	**`409`**	Not a permission problem — the command made no sense for the current state (pausing an already-paused player). The 5s poll lags reality; the frontend treats 409 as a resync cue, not an error.
`403` other	`403`	Genuine refusal (Premium, quota) — Spotify's wording passes through verbatim
`404`	retry, then `404`	Device id went stale — re-resolve and try once more
`429`	`429`	Rate limited
anything else	`502`	Upstream failed in an unanticipated way

> **Documentation describes intent; only observation describes behaviour**
>
> Spotify's docs state playback commands answer `204 No Content`. They answer `200` with a bare non-JSON command id and no `content-type` header:
>
> `POST /me/player/next → 200 b'HlY_NB7wNVBIfv5QLWEoCGylWiw'`
>
> An empty-body guard missed it, `.json()` raised, and every command returned **500** while succeeding on Spotify's side. Success parsing now keys off the header, not the spec.

05 Trust boundaries

Three zones. What matters is which secrets are allowed to cross which line.



The access token is the only credential allowed into the browser, and only for an hour.

A leaked access token dies within the hour and **cannot be renewed** — because the refresh token stayed behind. That single split is what turns a leak into an inconvenience rather than a compromise.

> **The unavoidable tradeoff**
>
> The Web Playback SDK runs in the browser, so the browser genuinely needs an access token — the one thing the design otherwise avoids. Time-boxing it to an hour, and keeping the refresh token server-side, is what makes that acceptable.

06 What CORS does — and does not

CORS is enforced **by the browser**, and governs one narrow thing: whether JavaScript on *site A* may read a response from *site B*. It is not a lock on the API.

✓ Stopped by CORS

A page on evil.com, open in your browser,
calling the API in the background.

Sends an Origin header → browser checks it → blocked.

✗ Not stopped by CORS

curl, a script, a phone, another app
on the same machine or network.

No Origin header → nothing to check → request proceeds.

Demonstrated on the dev machine

```
GET http://192.168.12.202:8000/spotify/token → 200 OK { "access_token": ... }
```

Ports publish on 0.0.0.0, so anyone on the same Wi-Fi can fetch a token.
CORS was never protecting it — the network binding was, and it was wide open.

The one-line fix

`ports: "8000:8000"`

binds 0.0.0.0 — every interface

reachable from the whole LAN

convenient if you want it open on your phone

`ports: "127.0.0.1:8000:8000"`

binds loopback only

"local only" becomes literally true

phone access is lost — that is the trade

> **Before ever distributing this**
>
> The blocker is not the token endpoint — it is `SPOTIFY_CLIENT_SECRET`. A shipped binary puts that secret on every user's disk, where it can be extracted and cannot be rotated without breaking every install. Public distribution requires **Authorization Code with PKCE**, which uses no secret at all.

Already right

- Client secret and refresh token are server-side only.
- OAuth `state` compared in constant time, in an `httponly` · `samesite=lax` · 10-minute cookie.
- Access tokens refresh 60s early, automatically.
- Request bodies are schema-validated — clients cannot set ids or timestamps.
- CORS origins, methods and headers named explicitly rather than wildcarded.